



Orientação a Objetos

Herança e Encapsulamento

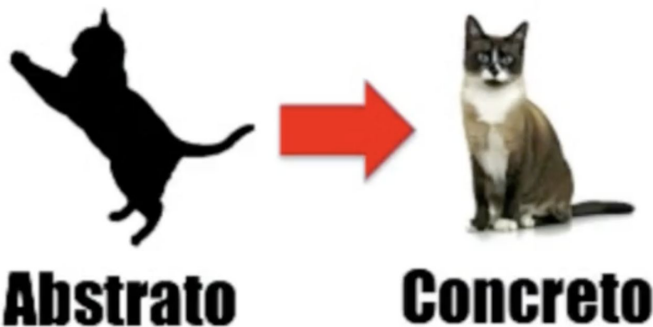
Relacionamento entre classes



**Criar um sistema orientado
a objetos tem a ver com
descobrir suas peças e
como elas se encaixam!**

Classe e Objeto

Já entendemos como as classes são as abstrações a partir das quais são criados os objetos!



Níveis de abstração

 **Felino**
Mais genérico



Mais específico
Gato de Rua 



**Todos são
abstrações!**

SOFTEx
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO



Abstrações parecidas

```
class Empregado{  
    nome?: string  
    idade?: number  
    salario?: number  
}
```



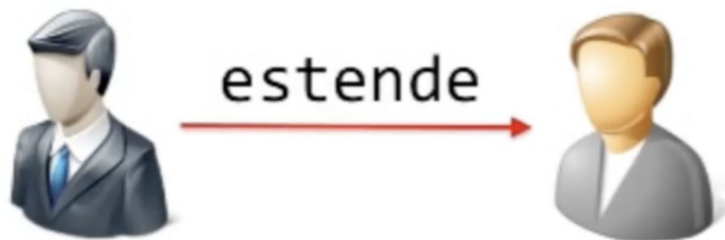
```
class Gerente{  
    nome?: string  
    idade?: number  
    salario?: number  
    bonus?: number  
}
```



**A herança é um recurso
da orientação a objetos
que permite que
classes especializem
ou generalizem outras
classes.**



Herança



A partir da herança é possível estender uma classe, adicionando atributos e modificando comportamento.

Herança



SUPERCLASSE



estende



SUBCLASSE

SOFTEx
PERNAMBUCO

 **Softex**

MINISTERIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO



Herança

```
class Empregado{  
    nome?: string  
    idade?: number  
    salario?: number  
}
```



*herda métodos e
atributos*

```
class Gerente extends Empregado{  
    bonus?: number  
}
```



Sobrescrita de Métodos

```
class Empregado{  
    liquido(salario: number){  
        return salario * 0.85  
    }  
}
```

precisa modificar o comportamento

```
class Gerente extends Empregado{  
    bonus?: number  
    //liquido() precisa adicionar o bonus  
}
```



Sobrescrita de Métodos

```
class Gerente extends Empregado{  
    bonus: number = 0  
    liquido(salario: number){  
        return salario * 0.85 + this.bonus  
    }  
}
```

**Uma classe pode
sobrescrever
métodos da
superclasse.**



Herança

```
class Empregado{  
    liquido(salario: number){  
        return salario * 0.85  
    }  
}
```

*Se mudarem as
taxas preciso
modificar em dois
locais!*

```
class Gerente extends Empregado{  
    bonus: number = 0  
    liquido(salario: number){  
        return salario * 0.85 + this.bonus  
    }  
}
```



Herança

```
class Gerente extends Empregado{  
    bonus: number = 0  
    liquido(salario: number){  
        return super.liquido(salario) + this.bonus  
    }  
}
```



Exercício da aula passada:

Exercício de Herança e Sobrescrita em TypeScript

O objetivo é modelar uma hiarquia de contas bancárias simples, utilizando herança para reutilizar o comportamento básico de uma conta e sobrescrever o método de saque na conta especializada.

Etapa	Classe	Atributos (Campos)	Métodos	Lógica Esperada
1	ContaCorrente (Super Classe)	saldo: number	depositar(valor: number) sacar(valor: number)	Adiciona <code>valor</code> ao <code>saldo</code> . Lógica Padrão: Só permite o saque se o <code>saldo</code> for maior ou igual ao <code>valor</code>
2	ContaEspecial (Sub Classe)	limite: number	sacar(valor: number)	Lógica Sobrescrita: Permite o saque se o <code>saldo</code> mais o <code>limite</code> for maior ou igual ao <code>valor</code>



Encapsulamento

```
const carro = new Carro()  
carro.velocidade = 100
```

Não é assim que muda
velocidade um carro! Eu
preciso chamar o
método acelerar() e
frear()!



Encapsulamento

Eu preciso permitir o acesso
ao atributo velocidade
somente dentro da classe...
Os métodos frear() e
acelerar() podem estar
acessíveis para todos!



Encapsulamento

**Os modificadores de
acesso podem ser
adicionados em
membros da classe
para definir quem tem
acesso a ela.**



Encapsulamento

private



Só pode ser
acessado por
membros da
própria classe!



Encapsulamento

public



Qualquer classe
pode acessar os
membros
públicos!



Encapsulamento

protected 

Somente subclasses
podem acessar!



property accessors

```
class Exemplo {  
    private _propriedade: tipo;  
  
    get propriedade(): tipo {  
        // Lógica do getter  
        return this._propriedade;  
    }  
  
    set propriedade(valor: tipo) {  
        // Lógica do setter  
        this._propriedade = valor;  
    }  
}
```



Exercícios

 Exercício - Refatoração - Conta bancária 

 Exercício - Refatoração 2 - Conta bancária 

 Exercício - Refatoração 3 - Pizzaria 

